Caching a read

Eleven questions, one in each format. Answer on the paper; the key starts on a new page.

Question 1 c-01 · beginner

A read finds the key in the cache. What is this called?

Tick one.

- ◯ **A** A hit
- ◯ **B** A miss
- ◯ **C** An eviction
- ◯ **D** A write-through

Question 2 c-02 · intermediate

Which of these make a cached value **stale**?

Tick every one that applies.

- ☐ **A** The order is updated in the database
- ☐ **B** The key's TTL is long and the data changes often
- ☐ **C** A second API instance reads the same key
- ☐ **D** The cache is restarted
- ☐ **E** An update skips the step that deletes the key

Question 3 c-03 · beginner

```
cache = {"42": "shipped"}
print(cache.get("43", "miss"))
```

Type exactly what this prints.

Answer: _____

Question 4 c-04 · intermediate

A cache answers 90 % of 1 000 reads at 2 ms each; the other reads go to a database at 40 ms.

What is the average time of a read, in milliseconds?

Answer: _____ ms

Question 5 c-05 · intermediate

```
1  def read(order_id):
2      hit = cache.get(order_id)
3      if hit:
4          return hit
5      row = db.load(order_id)
6      return row
```

Every read of the same order goes to the database. Write the number of the line that causes it.

Write the number of every faulty part.

Faulty: _____

Question 6 c-06 · beginner

On a miss the API reads the **(1)** _____ and then writes the value into the **(2)** _____.

Choose the word for each gap.

(1) _____ database · cache · queue

(2) _____ cache · database · log

Question 7 c-07 · intermediate

Put the steps of a read that misses in order.

Number them, first step to last.

- ☐ **A** The API answers the client
- ☐ **B** The cache answers that it does not have it
- ☐ **C** The API asks the cache for the key
- ☐ **D** The API writes the row into the cache
- ☐ **E** The API reads the row from the database

Question 8 c-08 · beginner

Sort each into what it holds.

Write each one's group: 1 Cache, 2 Database.

- ☐ Answers in a millisecond or two
- ☐ Survives a restart
- ☐ The record of what is true
- ☐ A copy that may be evicted

Question 9 c-09 · beginner

Match each term with what it means.

Write beside each number the letter it goes with.

- ☐ **1** TTL
- ☐ **2** Eviction
- ☐ **3** Hit rate
- ☐ **4** Warm-up

- **A** The share of reads the cache answers
- **B** How long a value may live in the cache
- **C** A value removed to make room
- **D** Filling the cache before the traffic comes

Question 10 c-10 · advanced

An interviewer asks *"How do you keep a cache from serving stale data?"* Answer as you would out loud.

Question 11 c-11 · advanced

After a deploy, p99 latency doubles for an hour and then returns to normal.

Work through a slow day in two steps.

(a)

What is the most likely cause?

Tick one.

- ◯ **A** The deploy emptied the cache
- ◯ **B** The database lost an index
- ◯ **C** Clients sent bigger requests

(b)

Name the practice that fills a cache before traffic arrives (one word).

Answer: _____

Answer key

Question 1

✓ **A** The value was in the cache, so the database is not asked.

✗ **B** A miss is the opposite: the key was not there, and the read goes on to the database.

✗ **C** An eviction removes a key to make room; it is not a read.

✗ **D** Write-through is a way of writing, to the cache and the store together.

> A hit is answered by the cache; a miss goes on to the store behind it.

Question 2

✓ **A** The cache still holds the old row until it is evicted or refreshed.

✓ **B** The longer a value lives, the longer it can lag behind the store.

✗ **C** Reading a value does not change it.

✗ **D** A restart empties the cache: the next read misses, but nothing stale is served.

✓ **E** Deleting the key on write is what keeps the cache honest.

> A value is stale when the store moved on and the cache was not told.

Question 3

✓ `miss`

✗ `None` `get` returns its second argument when the key is not there, not `None`.

Key `"43"` is not in the dictionary, so `get` returns the default it was given.

> `dict.get(key, default)` never raises: it returns the default on a miss.

Question 4

✓ **5.8 ms** (within 0.05 ms)

0.9 × 2 + 0.1 × 40 = 1.8 + 4 = 5.8 ms.

> The misses, though few, are most of the average.

Question 5

✓ Faulty: **6**

✗ **5** Loading on a miss is right; the problem is what happens after.

✓ **6** The row is returned but never put in the cache, so the next read misses again.

A read-through cache fills itself on the way back from a miss.

> A miss that does not fill the cache will miss again.

Question 6

(1) `database` The store of record answers what the cache cannot.

(2) `cache` Writing it back is what makes the next read a hit.

Question 7

✓ **C → B → E → D → A**

1. The API asks the cache for the key
2. The cache answers that it does not have it
3. The API reads the row from the database
4. The API writes the row into the cache
5. The API answers the client

Each step needs the one before it; the client is answered last, once the value is known.

Question 8

1 Cache A copy that may be evicted · Answers in a millisecond or two

2 Database The record of what is true · Survives a restart

The cache is a fast copy; the database is the truth it copies.

Question 9

✓ **1 → B, 2 → C, 3 → A, 4 → D**

Each term names one part of a cache's life.

Question 10

Delete or update the key whenever the data changes, and give every key a TTL as a safety net, so a missed delete is wrong for minutes, not forever. Where the data must never be stale, read it from the store.

A strong answer covers:

- Invalidate on write (delete or update the key)
- A TTL as a safety net
- Some reads should bypass the cache

Question 11

(a)

✓ **A** Every read missed until the cache had filled again.

✗ **B** That would not fix itself after an hour.

✗ **C** Nothing in the scenario changed the clients.

(b)

✓ `warm-up`, `warmup`

Warming the cache moves the misses to before the traffic.

> An empty cache is a slow cache; warm it before the traffic comes back.